| Ex.No.1 | INSTALL THE DATA ANALYSIS AND VISUALIZATION TOOL: PYTHON |
|---|---|

**1a. Aim:**

To download, install and explore the features of NumPy package.

# Problem Description

Python is an open-source object-oriented language. It has many features of which one is the wide range of external packages. There are a lot of packages for installation and use for expanding functionalities. These packages are a repository of functions in python script. NumPy is one such package to ease array computations. To install all these python packages we use the pip- package installer. Pip is automatically installed along with Python. We can then use pip in the command line to install packages from PyPI.

# NumPy

NumPy (Numerical Python) is an open-source library for the Python programming language. It is used for scientific computing and working with arrays.

Apart from its multidimensional array object, it also provides high-level functioning tools for working with arrays.

# Prerequisites

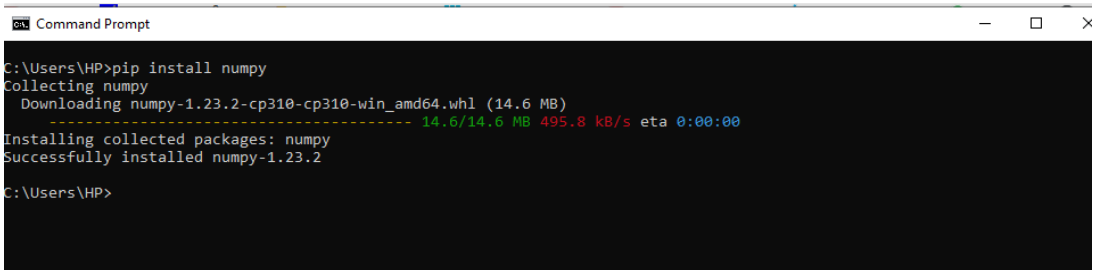
- Access to a terminal window/command line
- A user account with sudo privileges
- Python installed on your system

# Downloading and installing Numpy:

**Python NumPy** is a general-purpose array processing package that provides tools for handling n-dimensional arrays. It provides various computing tools such as comprehensive mathematical functions, linear algebra routines.Use the below command to install NumPy:

```
pip install numpy
```

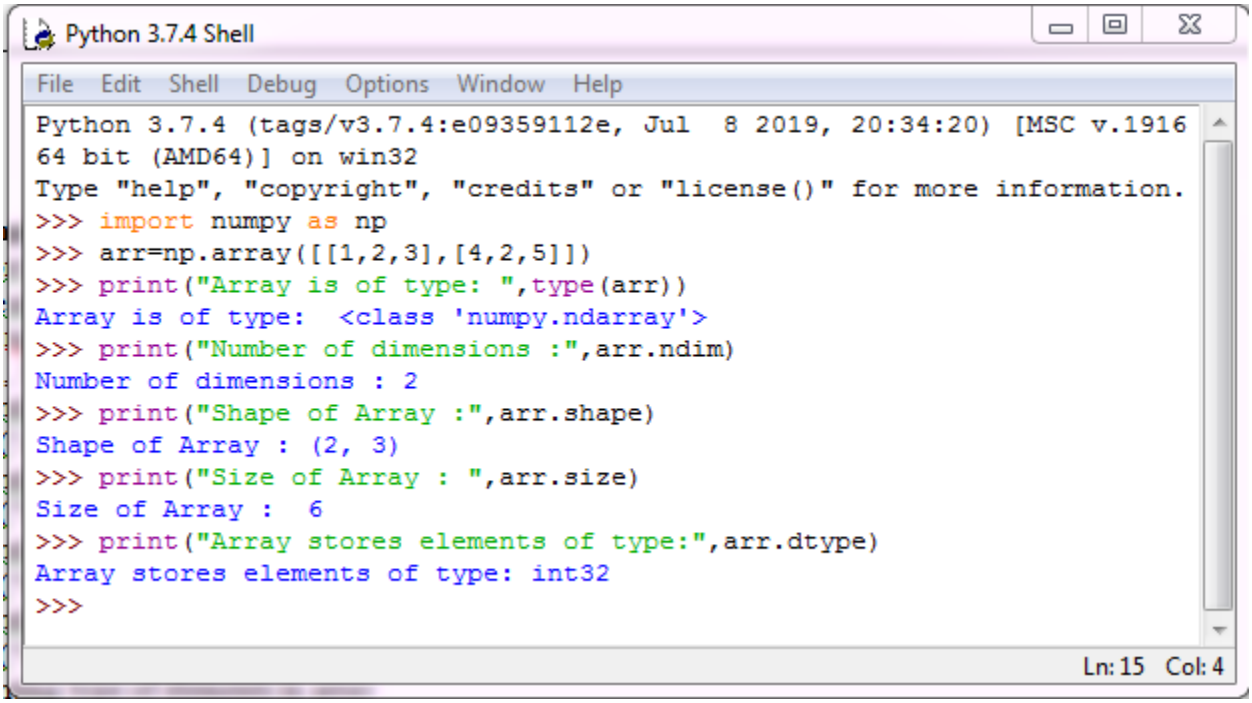
**OUTPUT:**




- **Sample python program using numpy**:

```
import numpy as np
# Creating array object
arr = np.array( [[ 1, 2, 3],
[ 4, 2, 5]] )
# Printing type of arr object
print("Array is of type: ", type(arr))
# Printing array dimensions (axes)
print("No. of dimensions: ", arr.ndim)
# Printing shape of array
print("Shape of array: ", arr.shape)
# Printing size (total number of elements) of array
print("Size of array: ", arr.size)
# Printing type of elements in array
print("Array stores elements of type: ", arr.dtype)
```

**OUTPUT:**




**RESULT:**

Thus, the NumPy package is downloaded, installed and the features are explored.

# 1 B) Aim:

To download, install and explore the features of Scipy package.

# Problem Description

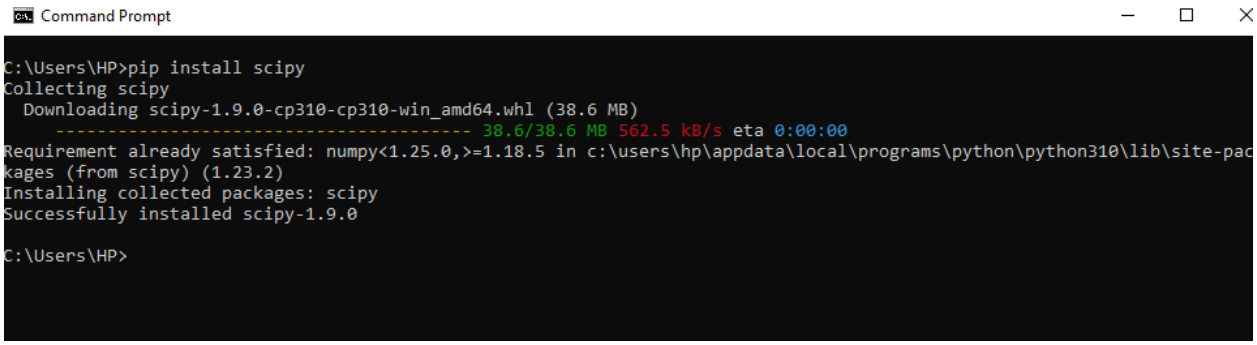
Scipy is a python library that is useful in solving many mathematical equations and algorithms. It is designed on the top of Numpy library that gives more extension of finding scientific mathematical formulae like Matrix Rank, Inverse, polynomial equations, LU Decomposition, etc. Using its high-level functions will significantly reduce the complexity of the code and helps in better analyzing the data.

# Downloading and Installing Scipy:

pip use the below command to install Scipy package on Windows:

## pip install scipy

**output**




# Sample python code using

# Scipy: Type the program

# in Jupyter notebook

```
from scipy import special
```

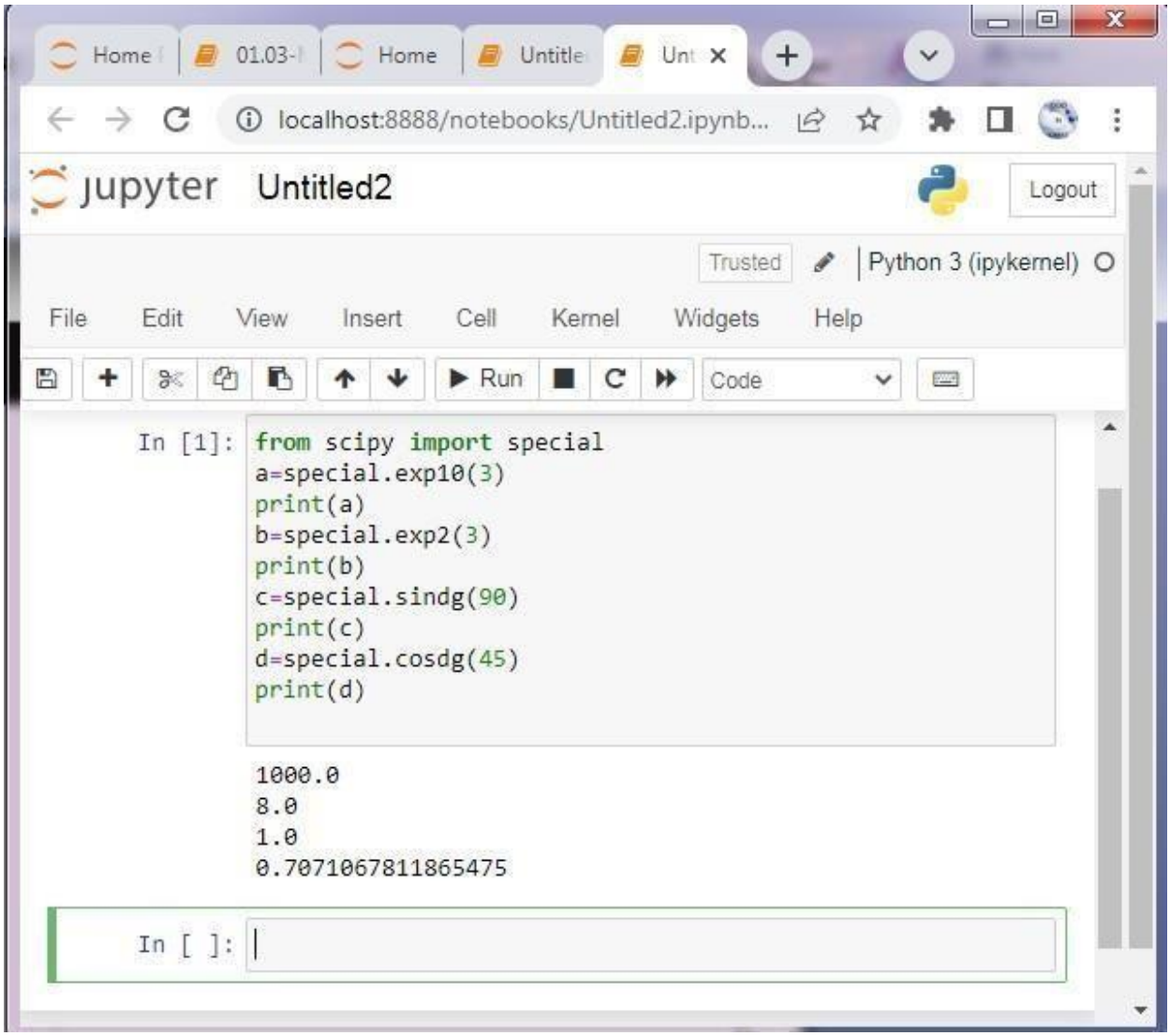
```
a = special.exp10(3)
print(a)
b = special.exp2(3)
print(b)
c = special.sindg(90)
print(c)

d = special.cosdg(45)

print(d)
```

---

# Output:




**RESULT**

Thus, the SciPy package is downloaded, installed and the features are explored.

1 C). **Aim** :

To download, install and explore the features of Panda packages.

# Problem Description

Pandas is one of the most popular open-source frameworks available for Python. It is among the fastest and most easy-to-use libraries for data analysis and manipulation. Pandas dataframes are some of the most useful data structures available in any library. It has uses in every data-intensive field, including but not limited to scientific computing, data science, and machine learning.

The library does not come included with a regular install of Python. To use it, you must install the Pandas framework separately.

# Installing Pandas on Windows

There are two ways of installing Pandas on Windows.

# Method #1: Installing with pip

It is a package installation manager that makes installing Python libraries and frameworks straightforward.

As long as you have a newer version of Python installed (> Python 3.4), pip will be installed on your computer along with Python by default.

However, if you're using an older version of Python, you will need to install pip on your computer before installing Pandas.

## Step #1: Launch Command Prompt

Press the Windows key on your keyboard or click on the Start button to open the start menu. Type **cmd**, and the Command Prompt app should appear as a listing in the start menu.

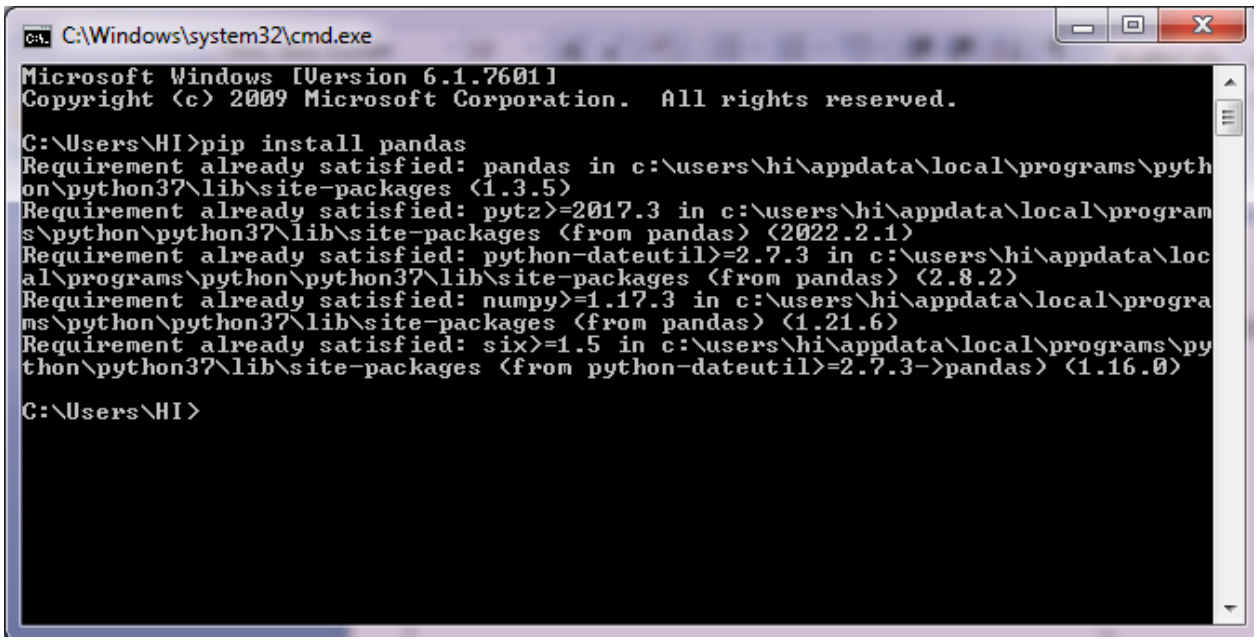
## Step #2: Enter the Required Command

After you launch the command prompt, the next step in the process is to type in the required command to initialize pip installation.

Enter the command

## pip install pandas

on the terminal. This should launch the pip installer. The required files will be downloaded, and Pandas will be ready to run on your computer.



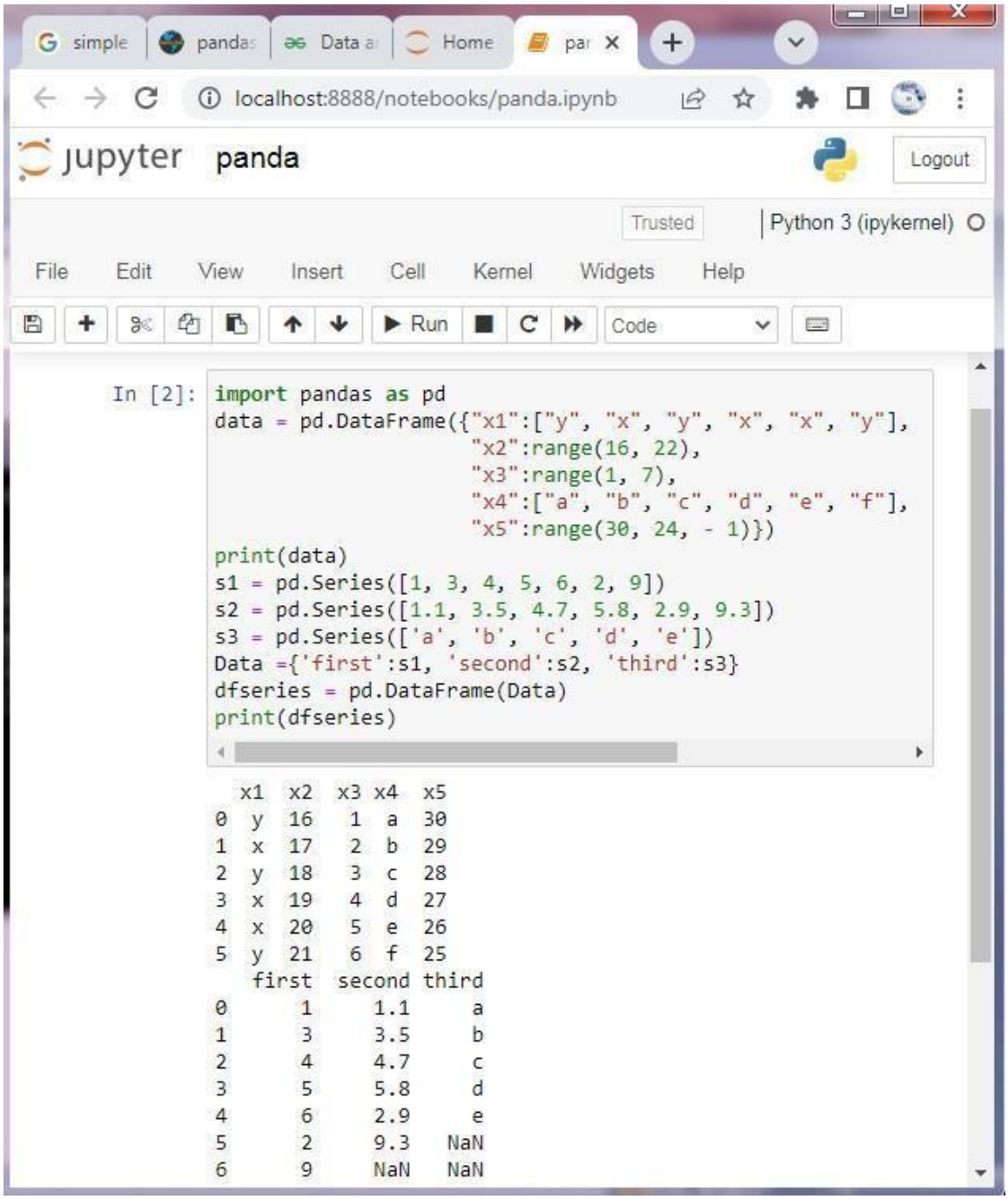

Panda package is successfully installed.

# Sample program

**// to be typed in Jupyter notebook**

```
import pandas as pd
data = pd.DataFrame({"x1":["y", "x", "y", "x", "x", "y"],
                     "x2":range(16, 22),
                     "x3":range(1, 7),
                     "x4":["a", "b", "c", "d", "e", "f"],
                     "x5":range(30, 24, - 1)})
print(data)
s1 = pd.Series([1, 3, 4, 5, 6, 2, 9])
s2 = pd.Series([1.1, 3.5, 4.7, 5.8, 2.9, 9.3])
s3 = pd.Series(['a', 'b', 'c', 'd', 'e'])
Data ={'first':s1, 'second':s2, 'third':s3}
```

dfseries = pd.DataFrame(Data)
print(dfseries)




## Result:

Thus, the Panda package is downloaded, installed and the features were explored.

| Ex.No.2 | PERFORM EXPLORATORY DATA ANALYSIS (EDA) ON WITH DATASETS LIKE EMAIL DATA SET |
|---|---|

# Aim :

To Perform EDA on dataset using packages and get insights from data.

## Procedure:

1. Import the dataset
2. View the head of the data
3. View the basic information of data and description of data
4. Find the unique value of data and verify the duplication of data
5. Plot a graph for unique value of dataset
6. Verify the presence of null value and replace then null value
7. Visualize the needed data

## Program:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Load the dataset
file_path = 'C:/Users/smily/Downloads/emails.csv'
# Adjust the path to your dataset file
df = pd.read_csv(file_path)

# Display the first few rows of the dataframe
print(df.head())
# Get basic info about the dataset
print(df.info())

# Get descriptive statistics for numeric columns
print(df.describe())

# Check for missing values
print(df.isnull().sum())
```

**OUTPUT:**

```
                                                   text  spam
0  Subject: naurally irresistible your corporate...     1
1  Subject: the stock trading gunslinger  fanny i...     1
2  Subject: unbelievable new homes made easy  im ...     1
3  Subject: 4 color printing special  request add...     1
4  Subject: do not have money , get software cds ...     1
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5728 entries, 0 to 5727
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   text    5728 non-null   object
 1   spam    5728 non-null   int64
dtypes: int64(1), object(1)
memory usage: 89.6+ KB
None
              spam
count  5728.000000
mean      0.238827
std       0.426404
min       0.000000
25%       0.000000
50%       0.000000
75%       0.000000
max       1.000000
text    0
```

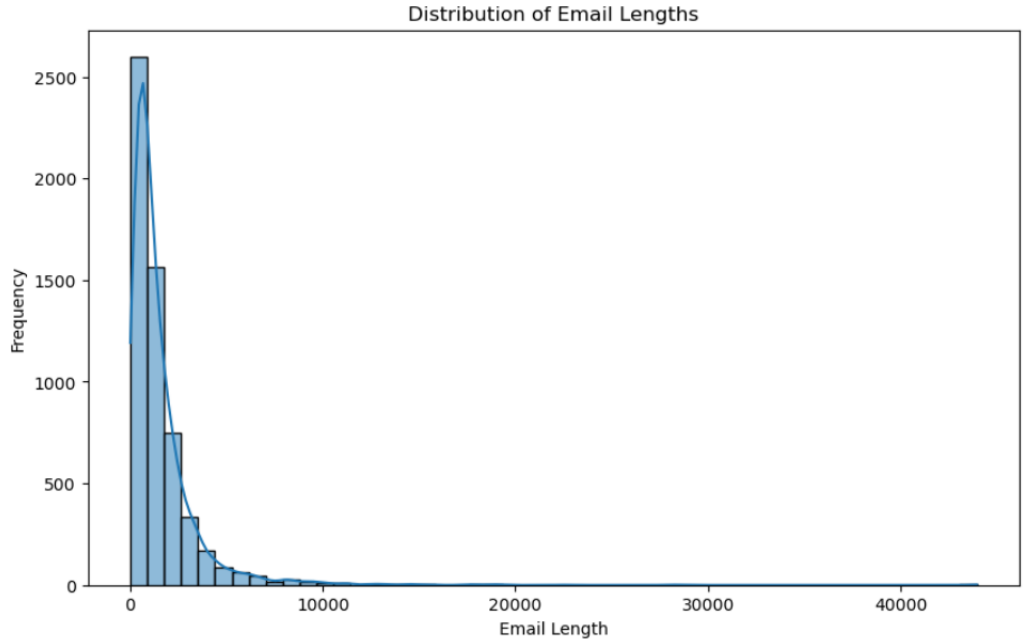
df['email_length'] = df['text'].apply(len)
print(df)

# Plot the distribution of email lengths
plt.figure(figsize=(10, 6))
sns.histplot(df['email_length'], bins=50, kde=True)
plt.title('Distribution of Email Lengths')
plt.xlabel('Email Length')
plt.ylabel('Frequency')
plt.show()

**OUTPUT:**

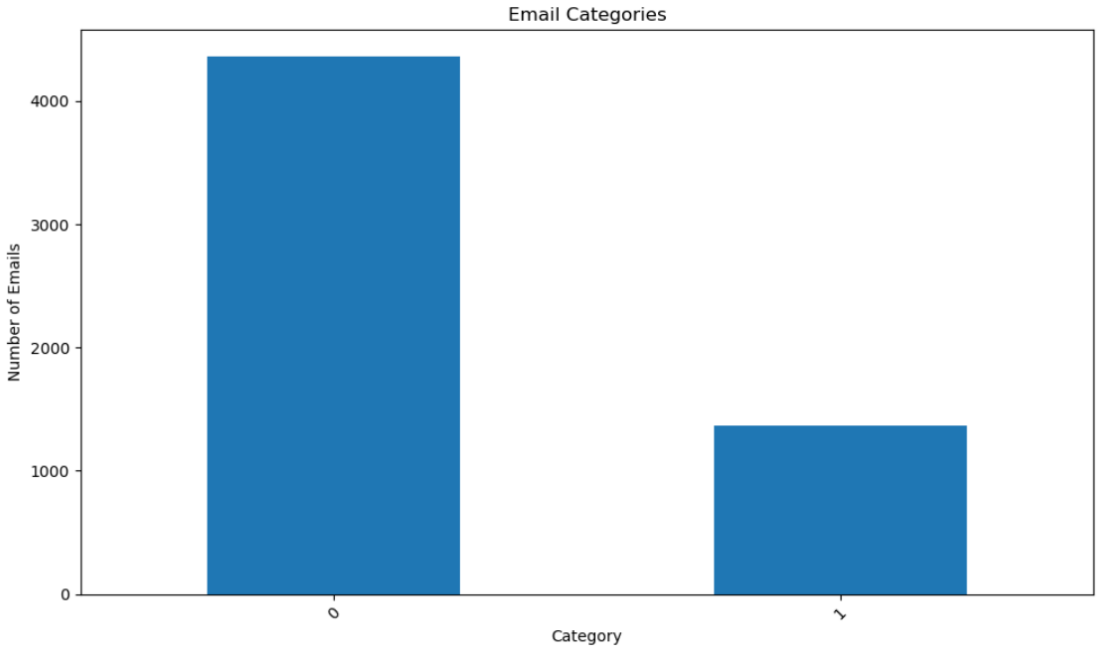
```
                                                   text  spam  email_length
0     Subject: naurally irresistible your corporate...     1          1484
1     Subject: the stock trading gunslinger  fanny i...     1           598
2     Subject: unbelievable new homes made easy  im ...     1           448
3     Subject: 4 color printing special  request add...     1           500
4     Subject: do not have money , get software cds ...     1           235
...                                                 ...   ...           ...
5723  Subject: re : research and development charges...     0          1189
5724  Subject: re : receipts from visit  jim ,  than...     0          1167
5725  Subject: re : enron case study update  wow ! a...     0          2131
5726  Subject: re : interest  david ,  please , call...     0          1060
5727  Subject: news : aurora 5 . 2 update  aurora ve...     0          2331

[5728 rows x 3 columns]
```




```
# Count emails by category
category_counts = df['spam'].value_counts()
# Plot email categories
plt.figure(figsize=(10, 6))
category_counts.plot(kind='bar')
plt.title('Email Categories')
plt.xlabel('Category')
plt.ylabel('Number of Emails')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

**OUTPUT:**




**Result:** Thus, the program to perform exploratory data analysis (EDA) on with datasets like email data set was executed.

| Ex.No.3 | USE NUMPY ARRAYS TO COMPUTE STATISTICS AND MANIPULATE NUMERICAL WEATHER DATA |
|---|---|

# Aim :

To write a program to work with Numpy for performing statistical analysis on weather data.

.

# Problem Description

**Arrays in NumPy:** NumPy's main object is the homogeneous multidimensional array.

- It is a table of elements (usually numbers), all of the same type, indexed by a tuple of positive integers.
- In NumPy dimensions are called *axes*. The number of axes is *rank*.
- NumPy's array class is called **ndarray**. It is also known by the alias **array**.

# Array creation:

There are various ways to create arrays in NumPy.

- For example, you can create an array from a regular Python **list** or **tuple** using the **array** function. The type of the resulting array is deduced from the type of the elements in the sequences.
- Often, the elements of an array are originally unknown, but its size is known. Hence, NumPy offers several functions to create arrays with **initial placeholder content**. These minimize the necessity of growing arrays, an expensive operation. **For example:** np.zeros, np.ones, np.full, np.empty, etc.
- To create sequences of numbers, NumPy provides a function analogous to range that returns arrays instead of lists.
- **arange:** returns evenly spaced values within a given interval. **step** size is specified.
- **linspace:** returns evenly spaced values within a given interval. **num** no. of elements are returned.
- **Reshaping array:** We can use **reshape** method to reshape an array. Consider an array with shape (a1, a2, a3, …, aN). We can reshape and convert it into another array with shape (b1, b2, b3, …, bM). The only required condition is: a1 x a2 x a3 … x aN = b1 x b2 x b3 … x bM . (i.e original size of array remains unchanged.)
- **Flatten array:** We can use **flatten** method to get a copy of array collapsed into **one dimension**. It accepts *order* argument. Default value is 'C' (for row-major order). Use 'F' for column major order.

**Program:**

```
import numpy as np

import pandas as pd

# Replace with your actual file path

df = pd.read_csv('C:/Users/smily/Downloads/weather_2012.csv', parse_dates=['Date/Time'])

print(df.head())
```

```
temps = df['Temp (C)'].to_numpy()

winds = df['Wind Spd (km/h)'].to_numpy()

vis = df['Visibility (km)'].to_numpy()

print("Temp → mean:", np.nanmean(temps), "min:", np.nanmin(temps), "max:", np.nanmax(temps), "std:",

np.nanstd(temps))

print("Wind Speed → mean:", np.nanmean(winds), "max:", np.nanmax(winds))

num_cloudy = np.sum(df['Weather'] == 'Cloudy')

print("Cloudy hours:", num_cloudy)
```

**OUTPUT:**

```
             Date/Time  Temp (C)  Dew Point Temp (C)  Rel Hum (%)  \
0 2013-01-01 00:00:00      -1.0                -1.7           95
1 2013-01-01 01:00:00      -2.0                -5.1           79
2 2013-01-01 02:00:00      -2.7                -6.0           78
3 2013-01-01 03:00:00      -5.6               -11.7           62
4 2013-01-01 04:00:00      -7.7               -12.6           68

   Wind Spd (km/h)  Visibility (km)  Stn Press (kPa)       Weather
0               35             6.4            99.89          Snow
1               35            16.1            99.93  Mainly Clear
2               28            19.3           100.08          Snow
3               30            25.0           100.21         Clear
4               35            19.3           100.32  Mainly Clear
Temp → mean: 8.761646174863388 min: -27.1 max: 33.0 std: 11.870081613012612
Wind Speed → mean: 14.91780510018215 max: 69
Cloudy hours: 1756
```

```
mask = (winds > 35) & (vis == 25)

print("Extreme windy & clear-vis hours:", np.sum(mask))

df['Month'] = df['Date/Time'].dt.month

pivot = df.pivot_table(index='Month', values='Temp (C)', aggfunc=['mean','min','max','count'])

print(pivot)

grouped = df.groupby('Weather')['Temp (C)'].agg(['mean','std','count'])

print(grouped.sort_values('count', ascending=False).head())
```

**OUTPUT:**

```
Extreme windy & clear-vis hours: 58
              mean      min      max    count
         Temp (C) Temp (C) Temp (C) Temp (C)
Month
1       -7.802419    -27.1      8.8      744
2       -4.225000    -16.9      6.1      696
3        3.121237    -16.2     25.5      744
4        7.009306     -3.0     27.8      720
5       16.237769      4.6     31.2      744
6       20.134028      9.5     33.0      720
7       22.790054     12.4     33.0      744
8       22.279301     10.5     32.8      744
9       16.484444      4.9     28.4      720
10      10.954973     -1.9     21.1      744
11       0.931389    -13.6     17.5      720
12      -3.306317    -16.0     11.9      744
                       mean        std  count
Weather
Mainly Clear     12.368336  12.606350   2116
Mostly Cloudy    10.686042  11.432611   2049
Cloudy            8.006435  10.879602   1756
Clear             6.281737  12.757931   1347
Snow             -5.010497   4.942024    362
```

**Result:**

Thus, the program to perform statistical analysis on weather data using Numpy arrays were executed.

| Ex.No.4 | DATA CLEANING ON MESSY CUSTOMER FEEDBACK DATA USING R |
|---|---|

**Aim:**

To explore various variable and column filters in R for cleaning messy customer feedback data.

## Procedure:

1. Import dplyr and stringr library
2. Create a dataset
3. Using filter functions rearrange and normalize the data
4. Using filter functions remove NA and apply conditions on data

**Progrma:**

```
library(dplyr)
library(stringr)
feedback_df <- data.frame(
  CustomerID = c(101, 102, 103, NA, 105, 106),
  Feedback = c("Good service", "bad Service!!", NA, "  excellent!! ", "Average ", ""),
  Rating = c(5, 1, NA, 5, 3, 2),
  Gender = c("Male", "Female", "F", "female", "MALE", "M"),
  stringsAsFactors = FALSE
)
print(feedback_df)
```

**OUTPUT:**

| | CustomerID | Feedback | Rating | Gender |
|---|---|---|---|---|
| 1 | 101 | Good service | 5 | Male |
| 2 | 102 | bad Service!! | 1 | Female |
| 3 | 103 | <NA> | NA | F |
| 4 | NA | excellent!! | 5 | female |
| 5 | 105 | Average | 3 | MALE |
| 6 | 106 | | 2 | M |

```
feedback_df <- feedback_df %>%
  rename(Customer_ID = CustomerID)
feedback_df <- feedback_df %>%
  mutate(
    Feedback = str_trim(Feedback),                  # Remove extra spaces
    Feedback = str_to_sentence(Feedback),           # Capitalize first letter
    Feedback = str_replace_all(Feedback, "[^[:alnum:] ]", "")  # Remove punctuation
  )
feedback_df <- feedback_df %>%
  mutate(
    Gender = str_to_lower(Gender),
    Gender = case_when(
      Gender %in% c("male", "m") ~ "Male",
      Gender %in% c("female", "f") ~ "Female",
      TRUE ~ "Other"
    )
  )
```

```
print(feedback_df)
```

**OUTPUT:**

| Customer_ID | Feedback | Rating | Gender |
|---|---|---|---|
| 1 101 | Good service | 5 | Male |
| 2 102 | Bad service | 1 | Female |
| 3 103 | <NA> | NA | Female |
| 4 NA | Excellent | 5 | Female |
| 5 105 | Average | 3 | Male |
| 6 106 | | 2 | Male |

```
feedback_df <- feedback_df %>%
  filter(!is.na(Customer_ID))
print(feedback_df)
```

**OUTPUT:**

| Customer_ID | Feedback | Rating | Gender |
|---|---|---|---|
| 1 101 | Good service | 5 | Male |
| 2 102 | Bad service | 1 | Female |
| 3 103 | <NA> | NA | Female |
| 4 105 | Average | 3 | Male |
| 5 106 | | 2 | Male |

```
feedback_df <- feedback_df %>%
  filter(!is.na(Feedback) & Feedback != "")
print(feedback_df)
```

**OUTPUT:**

| Customer_ID | Feedback | Rating | Gender |
|---|---|---|---|
| 1 101 | Good service | 5 | Male |
| 2 102 | Bad service | 1 | Female |
| 3 105 | Average | 3 | Male |

```
feedback_df <- feedback_df %>%
  filter(!is.na(Rating) & Rating >= 3)
print(feedback_df)
```

**OUTPUT:**

| Customer_ID | Feedback | Rating | Gender |
|---|---|---|---|
| 1 101 | Good service | 5 | Male |
| 2 105 | Average | 3 | Male |

## Result:

Thus, the R programs were written and executed to explore various variable and column filters in messy customer feedback data.

| Ex.No.5 | PERFORM ANALYSIS AND VISUALIZATIONS ON STUDENT TEST PERFORMANCE DATA USING GGPLOT |
|---|---|

# Aim:

To perform various analysis and visualization on student test performance using ggplot features.

**Procedure:**

1. Import dplyr and ggplot library
2. Create the dataframe
3. perform analysis using features of ggplot
3. Visualize the data using histogram, boxplot, violin plot and scatter plot

**Program:**

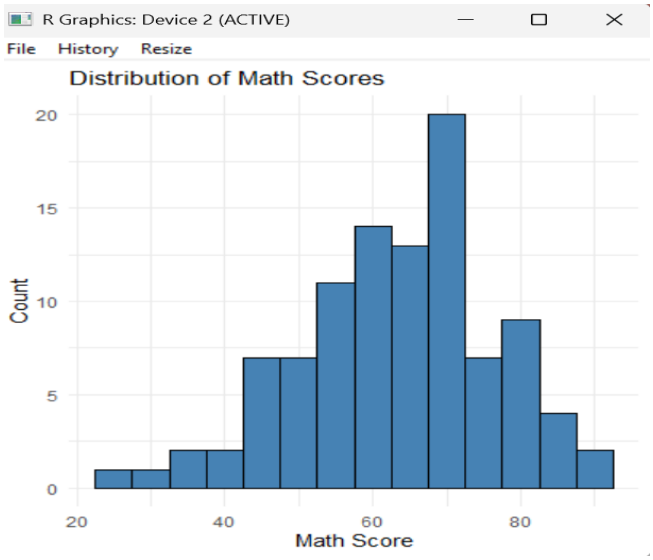
```
student_data <- data.frame(
  Gender = sample(c("Male", "Female"), 100, replace = TRUE),
  Math_Score = rnorm(100, mean = 65, sd = 15),
  Reading_Score = rnorm(100, mean = 70, sd = 10),
  Writing_Score = rnorm(100, mean = 68, sd = 12),
  Prep_Course = sample(c("Completed", "None"), 100, replace = TRUE)
)
print(head(student_data))
```

**OUTPUT:**

```
  Gender Math_Score Reading_Score Writing_Score Prep_Course
1 Female   79.78963      55.30700      66.20230        None
2   Male   54.49454      69.64660      78.83545   Completed
3   Male   48.62676      72.99901      80.08234   Completed
4   Male   45.22406      73.91637      42.74649        None
5   Male   71.87758      71.74030      67.33564   Completed
6 Female   61.00217      67.88042      66.28709        None
```

```
library(ggplot2)
ggplot(student_data, aes(x = Math_Score)) +
  geom_histogram(binwidth = 5, fill = "steelblue", color = "black") +
  theme_minimal() +
  labs(title = "Distribution of Math Scores", x = "Math Score", y = "Count")
```

**OUTPUT:**




```
ggplot(student_data, aes(x = Gender, y = Math_Score, fill = Gender)) +
  geom_boxplot() +
  theme_minimal() +
  labs(title = "Math Scores by Gender", x = "Gender", y = "Math Score")
```

**OUTPUT:**

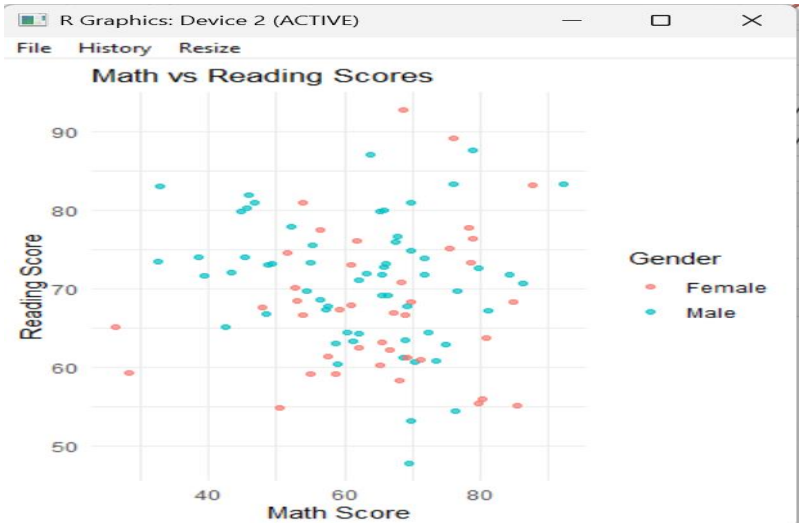
```
  ggplot(student_data, aes(x = Prep_Course, y = Reading_Score, fill = Prep_Course)) +
  geom_violin(trim = FALSE) +
  theme_minimal() +
  labs(title = "Reading Scores by Test Prep Course", x = "Test Preparation Course", y = "Reading
Score")
```

**OUTPUT:**




```
ggplot(student_data, aes(x = Math_Score, y = Reading_Score, color = Gender)) +
  geom_point(alpha = 0.7) +
  theme_minimal() +
  labs(title = "Math vs Reading Scores", x = "Math Score", y = "Reading Score")
```

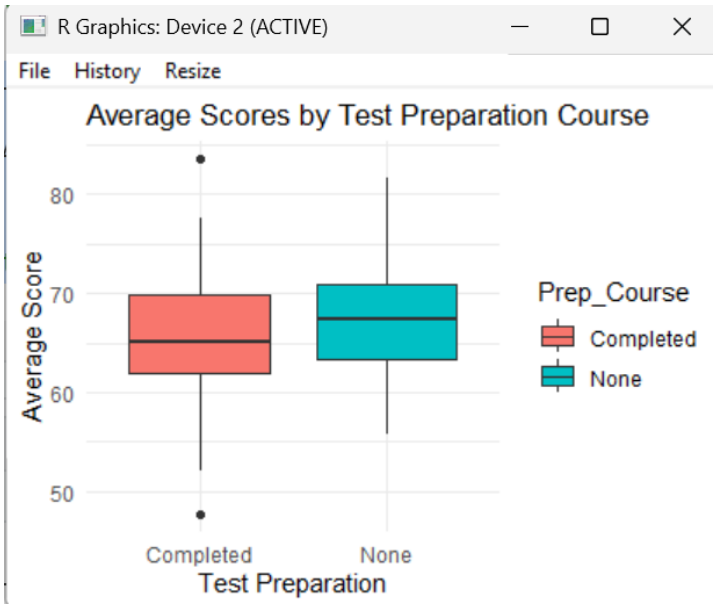
**OUTPUT:**



```
student_data$Average_Score <- rowMeans(student_data[, c("Math_Score", "Reading_Score",
"Writing_Score")])

ggplot(student_data, aes(x = Prep_Course, y = Average_Score, fill = Prep_Course)) +
  geom_boxplot() +
  theme_minimal() +
  labs(title = "Average Scores by Test Preparation Course", x = "Test Preparation", y = "Average Score")
```

**OUTPUT:**




## Result:

Thus, the R programs were written and executed to perform analysis and visualization on student test performance data using ggplot features.

| Ex.No.6 | USE PANDAS TO EXPLORE AND ANALYZE A MOCK DATASET OF A RETAIL STORE'S WEEKLY SALES |
|---|---|

# Aim:

To explore, analyze and visualize a mock retail store's weekly sales dataset using features of pandas.

# PROGRAM:

```
import pandas as pd
import numpy as np

# Set seed for reproducibility
np.random.seed(42)

# Create mock data
data = {
   "Week": pd.date_range(start="2023-01-01", periods=12, freq="W"),
   "Store_ID": np.random.choice([101, 102, 103], size=12),
   "Weekly_Sales": np.random.randint(10000, 30000, size=12),
   "Promotion": np.random.choice([0, 1], size=12),  # 1: Promotion Active, 0: No Promotion
   "Holiday_Week": np.random.choice([0, 1], size=12)
}

df = pd.DataFrame(data)
print(data)
```

**OUTPUT:**

```
{'Week': DatetimeIndex(['2023-01-01', '2023-01-08', '2023-01-15', '2023-01-22',
               '2023-01-29', '2023-02-05', '2023-02-12', '2023-02-19',
               '2023-02-26', '2023-03-05', '2023-03-12', '2023-03-19'],
              dtype='datetime64[ns]', freq='W-SUN'), 'Store_ID': array([103, 101, 103, 103, 101, 101, 103, 102, 103, 103, 103, 103]), 'Weekly_Sales': ar
ray([24423, 21363, 26023, 18322, 11685, 10769, 12433, 15311, 15051,
       16420, 27568, 29769]), 'Promotion': array([1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1]), 'Holiday_Week': array([1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0])}
```

```
print("First few rows:")
print(df.head())

print("\nSummary statistics:")
print(df.describe())

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())
```

**OUTPUT:**

```
First few rows:
        Week  Store_ID  Weekly_Sales  Promotion  Holiday_Week
0 2023-01-01       103         24423          1             1
1 2023-01-08       101         21363          0             0
2 2023-01-15       103         26023          1             1
3 2023-01-22       103         18322          0             1
4 2023-01-29       101         11685          0             0

Summary statistics:
                      Week    Store_ID  Weekly_Sales  Promotion  Holiday_Week
count                   12   12.000000     12.000000  12.000000     12.000000
mean   2023-02-08 12:00:00  102.416667  19094.750000   0.500000      0.583333
min    2023-01-01 00:00:00  101.000000  10769.000000   0.000000      0.000000
25%    2023-01-20 06:00:00  101.750000  14396.500000   0.000000      0.000000
50%    2023-02-08 12:00:00  103.000000  17371.000000   0.500000      1.000000
75%    2023-02-27 18:00:00  103.000000  24823.000000   1.000000      1.000000
max    2023-03-19 00:00:00  103.000000  29769.000000   1.000000      1.000000
std                    NaN    0.900337   6566.394903   0.522233      0.514929

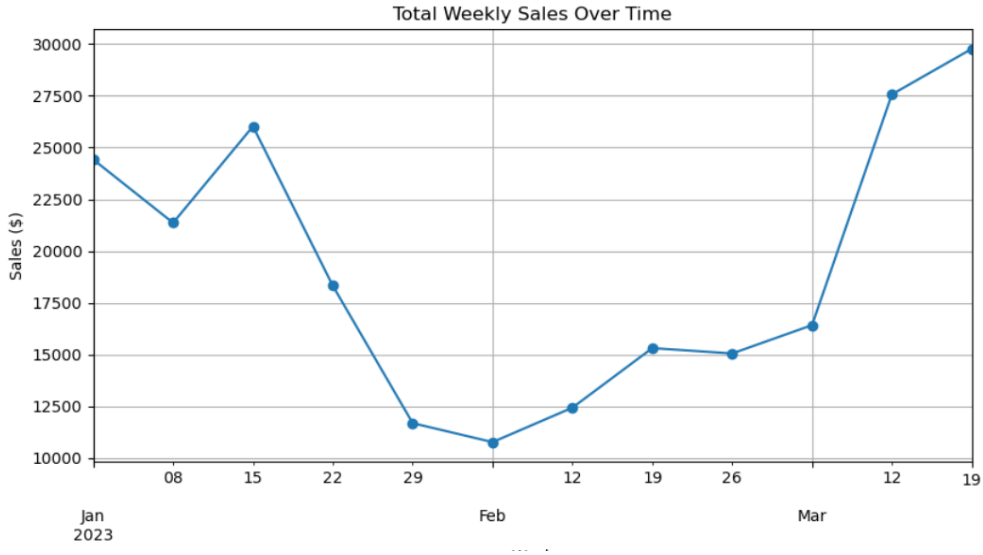
Data types:
Week            datetime64[ns]
Store_ID                 int32
Weekly_Sales             int32
Promotion                int32
Holiday_Week             int32
dtype: object

Missing values:
Week            0
Store_ID        0
Weekly_Sales    0
Promotion       0
Holiday_Week    0
dtype: int64
```

```
import matplotlib.pyplot as plt

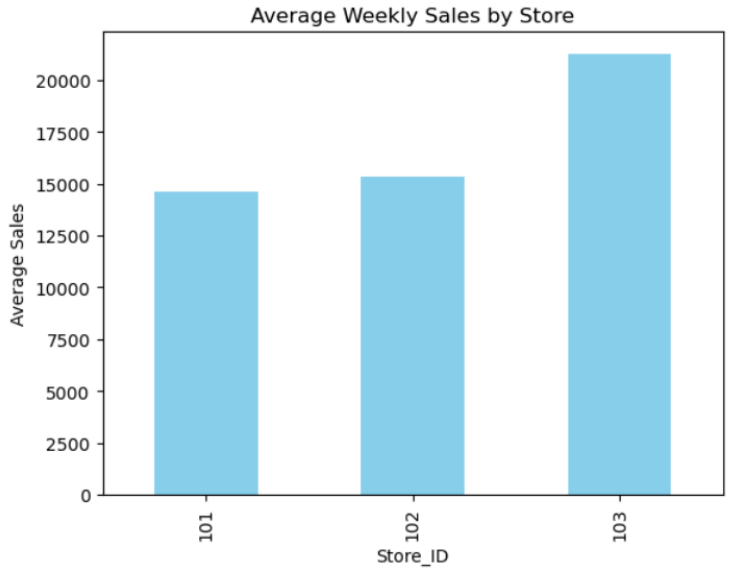
df.groupby("Week")["Weekly_Sales"].sum().plot(marker='o', figsize=(10, 5))
plt.title("Total Weekly Sales Over Time")
plt.xlabel("Week")
plt.ylabel("Sales ($)")
plt.grid(True)
plt.show()
```

**OUTPUT:**



```
df.groupby("Store_ID")["Weekly_Sales"].mean().plot(kind='bar', color='skyblue')
plt.title("Average Weekly Sales by Store")
plt.ylabel("Average Sales")
plt.show()
```

**OUTPUT:**




```
df.groupby("Promotion")["Weekly_Sales"].mean().plot(kind='bar', color=['red', 'green'])
plt.xticks([0, 1], ['No Promotion', 'Promotion Active'], rotation=0)
plt.title("Average Sales: Promotion vs No Promotion")
plt.ylabel("Average Weekly Sales")
plt.show()
```

**OUTPUT:**

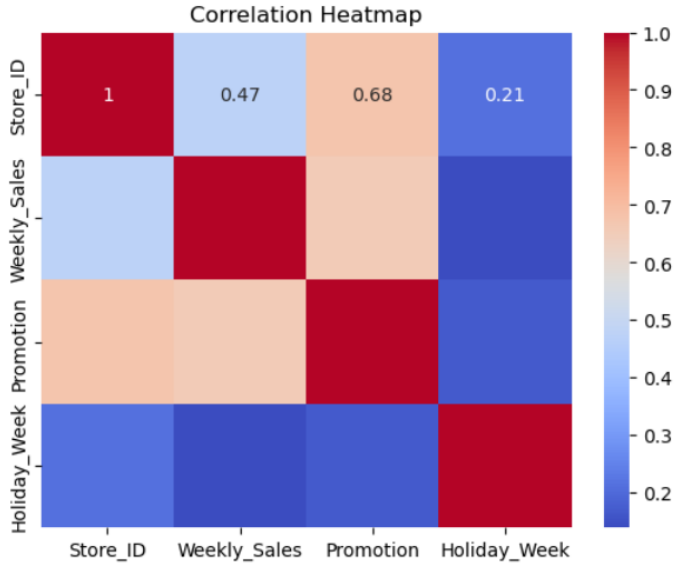


```
print("\nCorrelation Matrix:")
print(df.corr(numeric_only=True))

# Optional heatmap using seaborn
import seaborn as sns
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

**OUTPUT:**

```
Correlation Matrix:
              Store_ID  Weekly_Sales  Promotion  Holiday_Week
Store_ID      1.000000      0.472359   0.676716      0.212431
Weekly_Sales  0.472359      1.000000   0.654422      0.139077
Promotion     0.676716      0.654422   1.000000      0.169031
Holiday_Week  0.212431      0.139077   0.169031      1.000000
```




**Result:**

Thus, the python code was written and executed to explore, analyze and visualize a mock retail store's weekly sales dataset using features of pandas.

| Ex.No. 7 | USE SPATIAL DATASETS TO MAP VACCINATION RATES BY STATE WITH INTERACTIVE EFFECTS LIKE MOUSE ROLLOVER, TOOLTIPS, AND USER-SELECTED METRICS |
|---|---|

**Aim:**

To generate and visualize sample COVID-19 vaccination data for Indian states by creating bar charts, scatter plots, and heatmaps, in order to compare first dose, fully vaccinated, and booster dose rates across different regions.

**Procedure:**

1. **Import libraries** – pandas, seaborn, matplotlib, numpy.
2. **Define states & regions** – list of Indian states with their regional groupings.
3. **Generate sample vaccination data** – random values for first dose, fully vaccinated, and booster rates.
4. **Create Data Frame** – combine states, vaccination data, and regions.
5. **Sort states** – order by first dose rate (descending).
6. **Plot 1 (Bar chart)** – compare first dose vs fully vaccinated for top states.
7. **Plot 2 (Scatter plot)** – show relation between first dose and booster rates, colored by region.
8. **Plot 3 (Heatmap)** – display vaccination metrics for top states.

**PROGRAM:**

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 8)

# Indian states
states = [
    'Andhra Pradesh','Arunachal Pradesh','Assam','Bihar','Chhattisgarh',
    'Goa','Gujarat','Haryana','Himachal Pradesh','Jharkhand',
    'Karnataka','Kerala','Madhya Pradesh','Maharashtra','Manipur',
    'Meghalaya','Mizoram','Nagaland','Odisha','Punjab',
    'Rajasthan','Sikkim','Tamil Nadu','Telangana','Tripura',
    'Uttar Pradesh','Uttarakhand','West Bengal'
]

# Regions
regions = [
    'South','Northeast','Northeast','North','Central',
    'West','West','North','North','East',
    'South','South','Central','West','Northeast',
    'Northeast','Northeast','Northeast','East','North',
    'West','Northeast','South','South','Northeast',
    'North','North','East'
]
```

```
# Random vaccination data
np.random.seed(42)
df = pd.DataFrame({
   'state': states,
   'first_dose': np.random.uniform(40, 80, len(states)),
})
df['fully_vaccinated'] = df['first_dose'] * np.random.uniform(0.8, 0.95, len(df))
df['booster_rate'] = df['fully_vaccinated'] * np.random.uniform(0.5, 0.8, len(df))
df['region'] = regions
df = df.sort_values('first_dose', ascending=False)

# --- Plot 1: Bar comparison ---
top_states = df.head(15).melt(id_vars=['state'],
                        value_vars=['first_dose','fully_vaccinated'],
                        var_name='type', value_name='rate')

sns.barplot(data=top_states, x='state', y='rate', hue='type', palette='Set2')
plt.title('First Dose vs Fully Vaccinated (Top 15 States)')
plt.xticks(rotation=60)
plt.tight_layout()
plt.show()

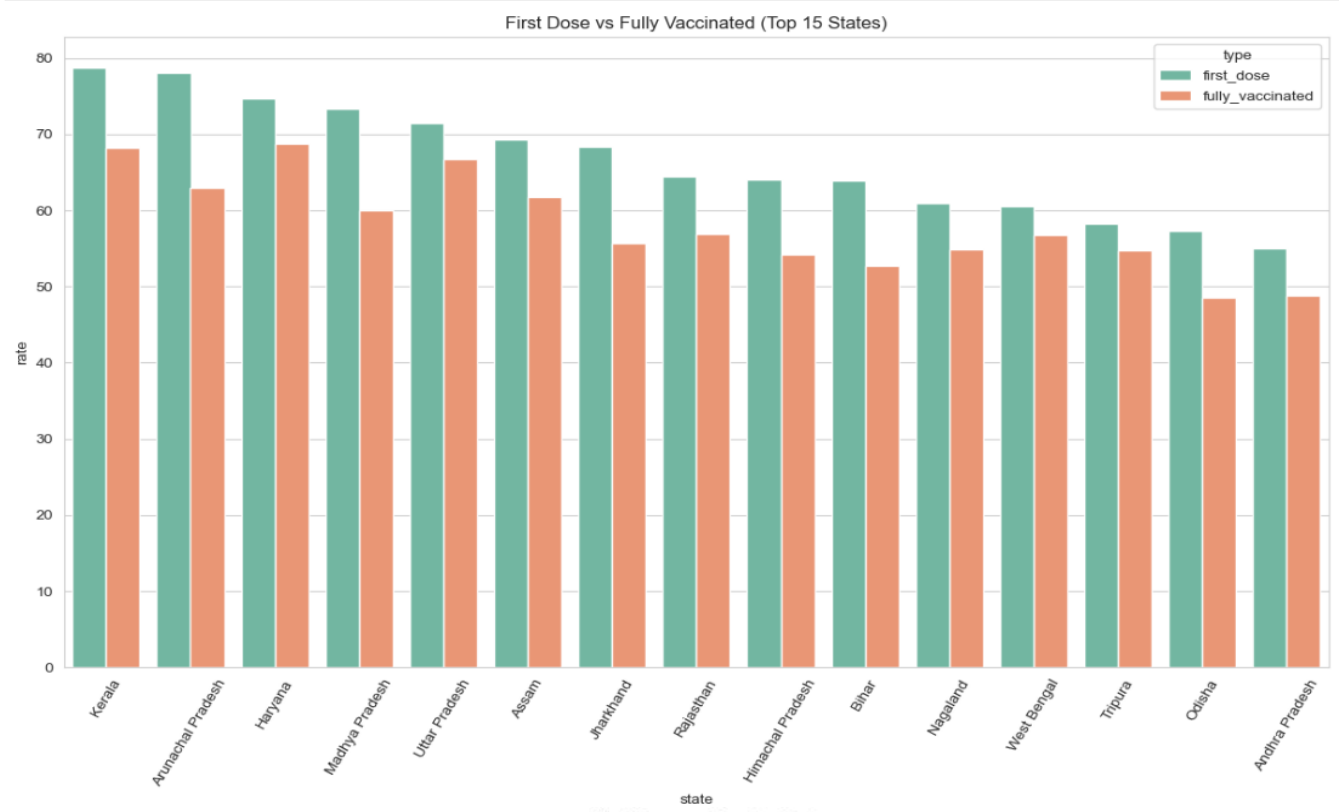
# --- Plot 2: Scatter ---
sns.scatterplot(data=df, x='first_dose', y='booster_rate',
             hue='region', s=100, palette='deep')

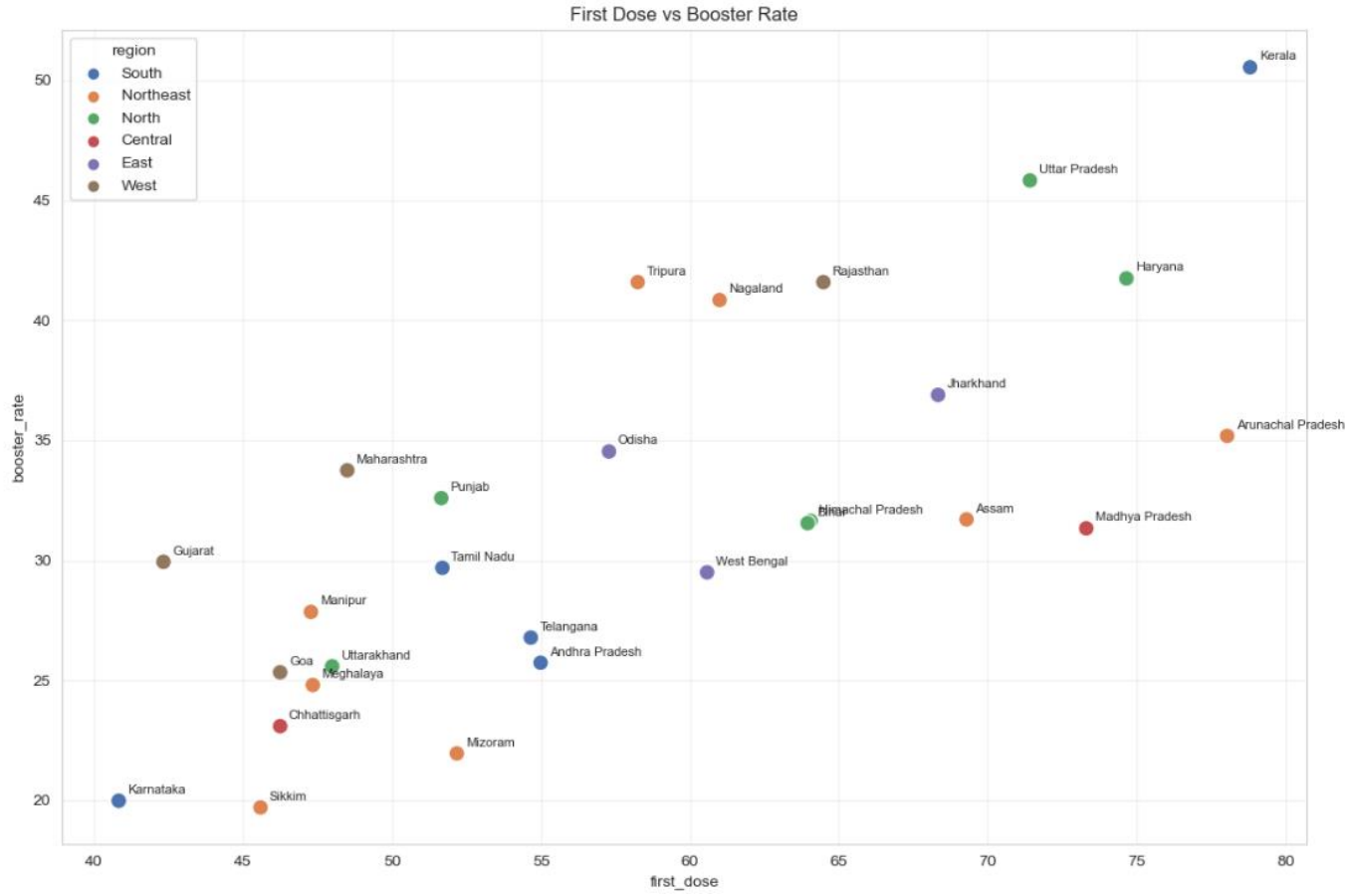
for x,y,label in zip(df['first_dose'], df['booster_rate'], df['state']):
   plt.text(x+0.3, y+0.3, label, fontsize=8)

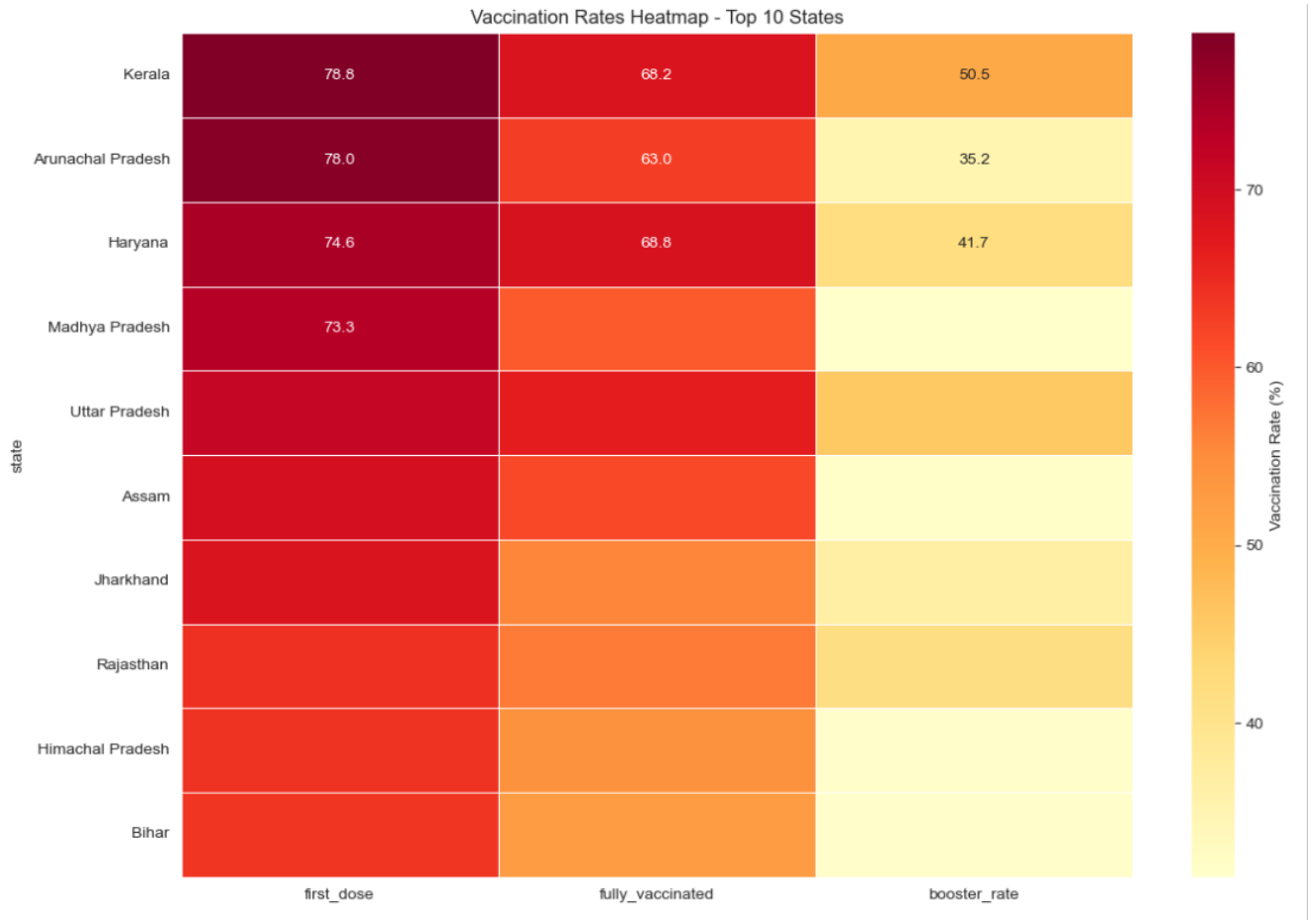
plt.title('First Dose vs Booster Rate')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# --- Plot 3: Heatmap ---
top_10 = df.head(10).set_index('state')[['first_dose','fully_vaccinated','booster_rate']]
sns.heatmap(top_10, annot=True, fmt='.1f', cmap='YlOrRd', linewidths=0.5,
          cbar_kws={'label': 'Vaccination Rate (%)'})
plt.title('Vaccination Rates Heatmap - Top 10 States')
plt.tight_layout()
plt.show()
```

**OUTPUT:**


First Dose vs Fully Vaccinated (Top 15 States)
type
first_dose
fully_vaccinated
rate
state
Kerala
Arunachal Pradesh
Haryana
Madhya Pradesh
Uttar Pradesh
Assam
Jharkhand
Rajasthan
Himachal Pradesh
Bihar
Nagaland
West Bengal
Tripura
Odisha
Andhra Pradesh


First Dose vs Booster Rate
region
South
Northeast
North
Central
East
West
booster_rate
first_dose
Kerala
Uttar Pradesh
Tripura
Nagaland
Rajasthan
Haryana
Jharkhand
Arunachal Pradesh
Odisha
Maharashtra
Punjab
Himachal Pradesh
Assam
Madhya Pradesh
Gujarat
Tamil Nadu
West Bengal
Manipur
Telangana
Goa
Uttarakhand
Andhra Pradesh
Meghalaya
Chhattisgarh
Mizoram
Karnataka
Sikkim




**Result:**

Thus, the python code was written and executed to explore, how data visualization can be used to compare vaccination progress across different states and regions in India.

| Ex.No. 8 | PERFORM EDA ON WINE QUALITY DATA SET |
|---|---|

**Aim:** To write a python program for EDA on Wine Quality Data Set.

**Procedure:**
1. Import library
2. Import wine dataset
3. Perform EDA to display information, description of data.
4. Analyse the content of alcohol consumption and visualize it

**Program:**

```
import pandas as pd
df_red = pd.read_csv("C:/Users/smily/Downloads/winequality-red.csv", delimiter=";")
df_white = pd.read_csv("C:/Users\smily/Downloads/winequality-white.csv", delimiter=";")
df_red.head()
#df_red.columns Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar','chlorides', 'free sulfur dioxide', 'total sulfur dioxide','density','pH','sulphates','alcohol', 'quality'],dtype='object')
df_red.iloc[100:110]
```

**OUTPUT:**

| | fixed acidity | volatile acidity | citric acid | residual sugar | chlorides | free sulfur dioxide | total sulfur dioxide | density | pH | sulphates | alcohol | quality |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **100** | 8.3 | 0.610 | 0.30 | 2.1 | 0.084 | 11.0 | 50.0 | 0.9972 | 3.40 | 0.61 | 10.2 | 6 |
| **101** | 7.8 | 0.500 | 0.30 | 1.9 | 0.075 | 8.0 | 22.0 | 0.9959 | 3.31 | 0.56 | 10.4 | 6 |
| **102** | 8.1 | 0.545 | 0.18 | 1.9 | 0.080 | 13.0 | 35.0 | 0.9972 | 3.30 | 0.59 | 9.0 | 6 |
| **103** | 8.1 | 0.575 | 0.22 | 2.1 | 0.077 | 12.0 | 65.0 | 0.9967 | 3.29 | 0.51 | 9.2 | 5 |
| **104** | 7.2 | 0.490 | 0.24 | 2.2 | 0.070 | 5.0 | 36.0 | 0.9960 | 3.33 | 0.48 | 9.4 | 5 |
| **105** | 8.1 | 0.575 | 0.22 | 2.1 | 0.077 | 12.0 | 65.0 | 0.9967 | 3.29 | 0.51 | 9.2 | 5 |
| **106** | 7.8 | 0.410 | 0.68 | 1.7 | 0.467 | 18.0 | 69.0 | 0.9973 | 3.08 | 1.31 | 9.3 | 5 |
| **107** | 6.2 | 0.630 | 0.31 | 1.7 | 0.088 | 15.0 | 64.0 | 0.9969 | 3.46 | 0.79 | 9.3 | 5 |
| **108** | 8.0 | 0.330 | 0.53 | 2.5 | 0.091 | 18.0 | 80.0 | 0.9976 | 3.37 | 0.80 | 9.6 | 6 |
| **109** | 8.1 | 0.785 | 0.52 | 2.0 | 0.122 | 37.0 | 153.0 | 0.9969 | 3.21 | 0.69 | 9.3 | 5 |

df_red.dtypes

**OUTPUT:**

```
fixed acidity           float64
volatile acidity        float64
citric acid             float64
residual sugar          float64
chlorides               float64
free sulfur dioxide     float64
total sulfur dioxide    float64
density                 float64
pH                      float64
sulphates               float64
alcohol                 float64
quality                   int64
dtype: object
```

df_red.describe()

**OUTPUT:**

| | fixed acidity | volatile acidity | citric acid | residual sugar | chlorides | free sulfur dioxide | total sulfur dioxide | density | pH | sulphates | alcohol | quality |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| count | 1599.000000 | 1599.000000 | 1599.000000 | 1599.000000 | 1599.000000 | 1599.000000 | 1599.000000 | 1599.000000 | 1599.000000 | 1599.000000 | 1599.000000 | 1599.000000 |
| mean | 8.319637 | 0.527821 | 0.270976 | 2.538806 | 0.087467 | 15.874922 | 46.467792 | 0.996747 | 3.311113 | 0.658149 | 10.422983 | 5.636023 |
| std | 1.741096 | 0.179060 | 0.194801 | 1.409928 | 0.047065 | 10.460157 | 32.895324 | 0.001887 | 0.154386 | 0.169507 | 1.065668 | 0.807569 |
| min | 4.600000 | 0.120000 | 0.000000 | 0.900000 | 0.012000 | 1.000000 | 6.000000 | 0.990070 | 2.740000 | 0.330000 | 8.400000 | 3.000000 |
| 25% | 7.100000 | 0.390000 | 0.090000 | 1.900000 | 0.070000 | 7.000000 | 22.000000 | 0.995600 | 3.210000 | 0.550000 | 9.500000 | 5.000000 |
| 50% | 7.900000 | 0.520000 | 0.260000 | 2.200000 | 0.079000 | 14.000000 | 38.000000 | 0.996750 | 3.310000 | 0.620000 | 10.200000 | 6.000000 |
| 75% | 9.200000 | 0.640000 | 0.420000 | 2.600000 | 0.090000 | 21.000000 | 62.000000 | 0.997835 | 3.400000 | 0.730000 | 11.100000 | 6.000000 |
| max | 15.900000 | 1.580000 | 1.000000 | 15.500000 | 0.611000 | 72.000000 | 289.000000 | 1.003690 | 4.010000 | 2.000000 | 14.900000 | 8.000000 |

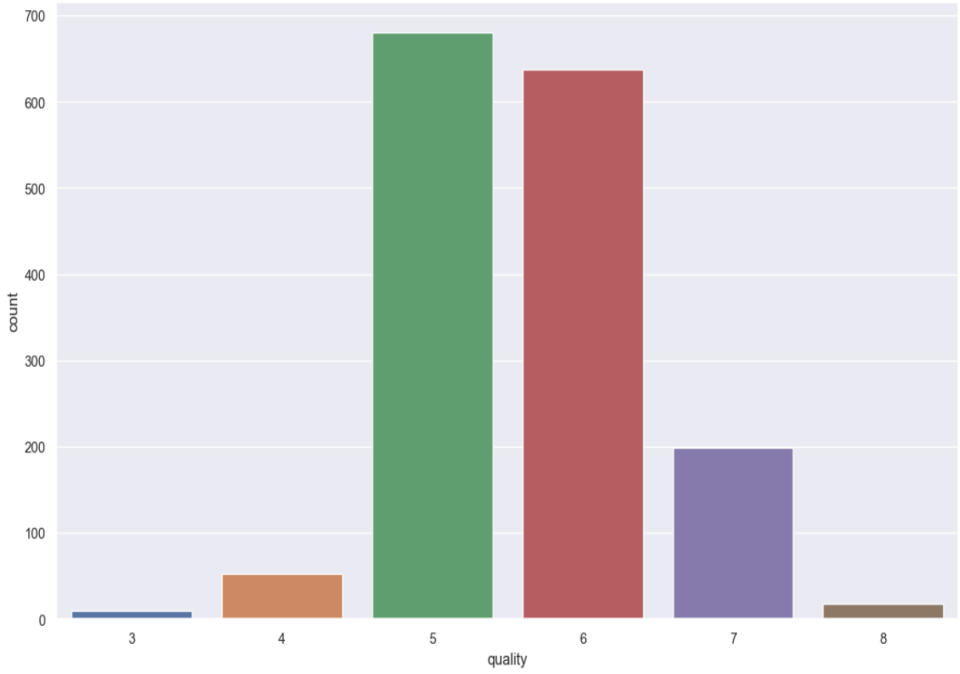
df_red.info()

**OUTPUT:**

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
 #    Column                Non-Null Count  Dtype
---   ------                --------------  -----
 0    fixed acidity         1599 non-null   float64
 1    volatile acidity      1599 non-null   float64
 2    citric acid           1599 non-null   float64
 3    residual sugar        1599 non-null   float64
 4    chlorides             1599 non-null   float64
 5    free sulfur dioxide   1599 non-null   float64
 6    total sulfur dioxide  1599 non-null   float64
 7    density               1599 non-null   float64
 8    pH                    1599 non-null   float64
 9    sulphates             1599 non-null   float64
 10   alcohol               1599 non-null   float64
 11   quality               1599 non-null   int64
dtypes: float64(11), int64(1)
memory usage: 150.0 KB
```

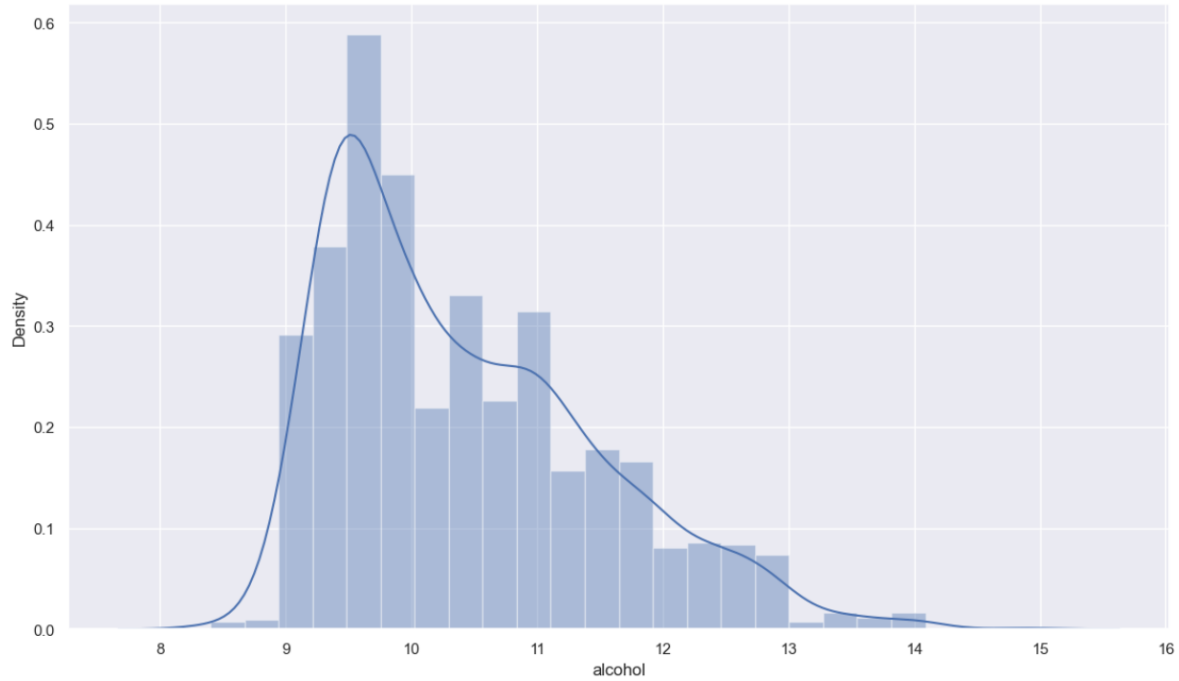
import seaborn as sns
sns.set(rc={'figure.figsize': (14, 8)})
sns.countplot(df_red, x="quality")

**OUTPUT:**



sns.distplot(df_red['alcohol'])

**OUTPUT:**




from scipy.stats import skew

skew(df_red['alcohol'])

**OUTPUT:**

```
0.8600210646566755
```

**Result:**

Thus, the program to execute the EDA on wine dataset was executed successfully.

| Ex.No.9 | CASE STUDY - USE THE DIABETES DATA SET FROM PIMA INDIAN'S DIABETES |
|---|---|

**Aim:**

To perform Univariate analysis like Frequency, Mean, Median, Mode, Variance, Standard Deviation, Skewness and Kurtosis in pima Indians dataset.

**Program:**

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from scipy.stats import skew
from scipy.stats import kurtosis
import statistics

df = pd.read_csv('C:/Users/HP/OneDrive/Desktop/RESEARCH/Data/Pima2.csv')
print('THE SHAPE OF THE DATASET IS \n',df.shape)
print('THE DATA TYPES ARE :\n',df.dtypes)
print('THE DESCRIPTION OF THE DATASET IS:\n',df.describe().T)
df.plot(kind='density', subplots=True, layout=(3,3), sharex=False)

print('THE FREQUENCY OF PREGNANCIES IS:\n',df['npreg'].value_counts())
preg = np.array(df['npreg'])
print('MEAN                    :',statistics.mean(preg))
print('MEDIAN                  :',statistics.median(preg))
print('MODE                    :',statistics.multimode(preg))
print('VARIANCE                :',statistics.variance(preg))
print('sTANDARS DEVIATION      :',statistics.stdev(preg))
if (skew(preg)>0) :
   print("POSITIVE  SKEWNESS \n")
elif (skew(preg)<0) :
   print("NEGATIVE  SKEWNESS \n")
else  :
   print("NO  SKEWNESS \n")
print('KURTOSIS',kurtosis(preg))



print('THE FREQUENCY OF GLOCOSE IS:\n',df['glu'].value_counts())
G = np.array(df['glu'])
print('MEAN                    :',statistics.mean(G))
print('MEDIAN                  :',statistics.median(G))
print('MODE                    :',statistics.multimode(G))
print('VARIANCE                :',statistics.variance(G))
print('sTANDARS DEVIATION      :',statistics.stdev(G))
if (skew(G)>0) :
   print("POSITIVE  SKEWNESS \n")
elif (skew(G)<0) :
```

```
   print("NEGATIVE  SKEWNESS \n")
else  :
   print("NO  SKEWNESS \n")
print('KURTOSIS',kurtosis(G))
```

**Output of above coding:**

```
THE SHAPE OF THE DATASET IS
 (300, 9)
THE DATA TYPES ARE :
 Unnamed: 0      int64
npreg          int64
glu            int64
bp           float64
skin         float64
bmi          float64
ped          float64
age            int64
type          object
dtype: object
THE DESCRIPTION OF THE DATASET IS:
             count        mean        std  ...      50%       75%       max
Unnamed: 0  300.0  150.500000  86.746758  ...  150.500  225.25000  300.000
npreg       300.0    3.786667   3.306195  ...    3.000    6.00000   14.000
glu         300.0  123.743333  30.011549  ...  121.000  142.00000  199.000
bp          287.0   72.320557  11.704477  ...   72.000   80.00000  114.000
skin        202.0   29.153465  11.682415  ...   29.000   36.00000   99.000
bmi         297.0   32.052862   6.492933  ...   32.000   36.50000   52.900
ped         300.0    0.435657   0.294600  ...    0.336    0.58675    2.288
age         300.0   33.096667  11.578176  ...   29.000   40.00000   72.000

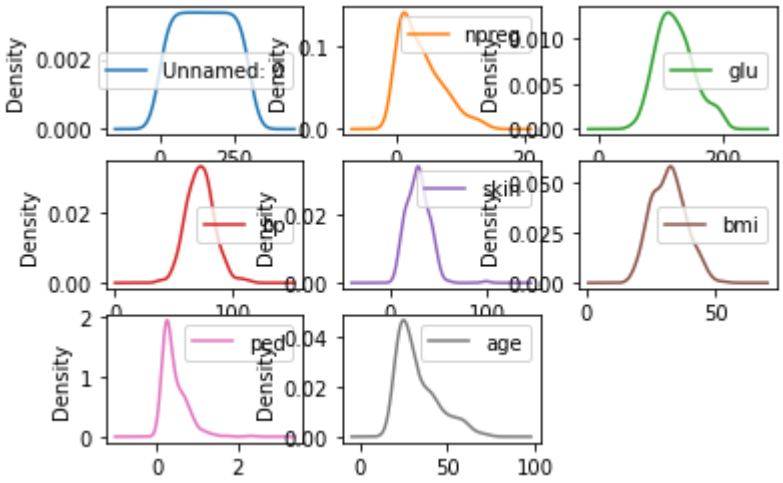
[8 rows x 8 columns]

THE FREQUENCY OF PREGNANCIES IS:
 1     51
0      44
2      41
4      35
3      26
5      22
6      19
7      17
8      15
9       9
10      8
12      6
13      3
14      2
11      2
Name: npreg, dtype: int64
MEAN                    : 3
```

MEDIAN : 3.0
MODE : [1]
VARIANCE : 10
STANDARS DEVIATION : 3.1622776601683795
POSITIVE SKEWNESS

KURTOSIS 0.20897562301271444
THE FREQUENCY OF GLOCOSE IS:
100 8
125 8
111 6
95 6
139 6
..
152 1
149 1
135 1
198 1
56 1
Name: glu, Length: 108, dtype: int64
MEAN : 123
MEDIAN : 121.0
MODE : [100, 125]
VARIANCE : 900
sTANDARS DEVIATION : 30.0
POSITIVE SKEWNESS

KURTOSIS -0.19604758469298522




**Result:**

Thus, the univariate analysis like Frequency, Mean, Median, Mode, Variance, Standard Deviation, Skewness and Kurtosis in pima Indians dataset was executed successfully.